

JDBC

Java DataBase Connectivity

Omar Rifi

Introduction

- JDBC offers an API (Application Programming Interface) to access any DBMS (Database Management System) that is conform to standard SQL.
- A Java application is able to access any DBMS in a generic way regardless of its producer.
- The DBMS producer should provide a JDBC Driver to allow Java applications to access that DBMS.
- A JDBC driver gives out the connection to the database and implements the protocol for transferring the query and result between client and database.

Driver Loading & Database Connection

- A driver must be registered with the `DriverManager` of JDBC. Example of loading the driver for Apache Derby DB:

```
DriverManager.registerDriver(new org.apache.derby.jdbc.ClientDriver());
```

- But, when a driver class is loaded, it creates an instance of itself and register it with the `DriverManager`. Therefore, it is sufficient to just load the class as follows:

```
Class.forName("org.apache.derby.jdbc.ClientDriver");
```

- Once the driver is loaded, you can open a connection to a database as follows:

```
Connection con = DriverManager.getConnection(url, user, password);
```

DBs are identified via a URL (Uniform Resource Locator). For a derby DB, it is of the form `jdbc:derby://localhost/sample`

- JDBC 4 in JDK 6, offers Autoloading of JDBC drivers.

Prepare/Execute Queries

- Once a connection is established, you can create and execute SQL queries (`Statement`).
- 3 types of statement:
 - `Statement` : simple queries (static SQL)
 - `PreparedStatement` : precompiled queries (dynamic SQL). More efficient than `Statement`.
 - `CallableStatement` : enclose SQL Stored Procedures.
- 3 types of execution
 - `executeQuery` : for queries returning results (SELECT).
 - `executeUpdate` : for queries like (INSERT, UPDATE, ...).
 - `execute` : when the query type is unknown.

Example

- Statement creation

```
Statement stmt = con.createStatement();
```

- Query execution

```
String query = "select first_name,last_name,age,dob from person";  
ResultSet rs = stmt.executeQuery(query);
```

- `executeQuery` returns an object of type `ResultSet` which contains the result of the query and can be traversed row by row.

```
while (rs.next()) {  
    // read data from the current row  
}
```

The first call to `rs.next()` places the `ResultSet` on the first row. Each subsequent call, advances the `ResultSet` to the next row.

Reading Data

Suppose the table `person` is created as follows:

```
create table person (  
    first_name varchar(45),  
    last_name varchar(45),  
    age integer,  
    dob date  
)
```

You can access element in a row as follows:

```
while (rs.next()) {  
    String first = rs.getString(1);    // rs.getString("first_name");  
    String last = rs.getString(2);    // rs.getString("last_name");  
    int age = rs.getInt(3);           // rs.getInt("age");  
    java.sql.Date date = rs.getDate(4); // rs.getDate("dob");  
}  
  
rs.close();
```

Equivalence between Java and SQL types

SQL Type	Java Type	Method
CHAR	String	getString
VARCHAR	String	getString
BOOLEAN or BIT	boolean	getBoolean
SMALLINT	short	getShort
INTEGER	int	getInt
REAL	float	getFloat
FLOAT	double	getDouble
DOUBLE	double	getDouble
DATE	java.sql.Date	getDate
TIME	java.sql.Time	getTime

Handling NULL values

What happens when method `getXXX()` is applied on a SQL NULL value?

The method will return

- null if `getXXX()` returns an object (e.g. `getString()`, `getDate()`).
- 0 if `getXXX()` returns a numeric type (e.g. `getInt()`, `getDouble()`).
- false for `getBoolean()`.

You can use the method `wasNull()` in `ResultSet` to distinguish between SQL value 0 (or false) and SQL value NULL.

```
int age = rs.getInt();
if (age == 0 && rs.wasNull()) {
    // age has NULL value in table person
}
```

Prepared Statement

- Used to define parameterized queries.

```
String query = "select * from person where age = ? " +
               "and last_name = ?";
PreparedStatement ps = con.prepareStatement(query);
```

- Setting the parameter

```
ps.setInt(1, 40);
ps.setString(2, "...");
```

- Invocation

```
ResultSet rs = ps.executeQuery();
```

Access to metadata

Consider the following example in which an unknown query is executed

```
if (stmt.execute(cmd)) { // returns true if (SELECT) false otherwise
    ResultSet rs = stmt.getResultSet();
    //ResultSet Traversal, need to access ResultSet metadata
    rs.close();
}
```

The method `getMetadata()` of class `Connection` returns a `DatabaseMetaData` object from which you can access information about tables, columns of tables, etc.

The method `getMetadata()` of class `ResultSet` returns a `ResultSetMetaData` object from which you can get

- Number of columns: `getColumnCount()`.
- Name of a column: `getColumnName()`.
- Type of a column: `getColumnType()`.

Handling Transaction

An example that shows how to manage transactions.

```
try {
    // Default value true
    con.setAutoCommit(false);

    // Queries that should be executed as one
    stmt.executeUpdate("...");
    stmt.executeUpdate("...");
    ...

    // Validate the transaction
    con.commit()
} catch (SQLException e) {
    // Cancel the transaction
    con.rollback();
}
```

About SQLException

SQLException defines the following methods:

- `getSqlState()` : returns a state code SQL Standard.
- `getErrorCode` : returns an error code vendor-specific.
- `getNextException` : returns the next Exception.

```
catch (SQLException e) {
    while (e != null) {
        System.out.println("SQL Exception");
        System.out.println(e.getMessage());
        System.out.println("ANSI-92 SQL State : "+e.getSQLState());
        System.out.println("Vendor error code : "+e.getErrorCode());
        e = e.getNextException();
    }
}
```

Notice that you can access SQL Warnings using the method `getWarnings()` from a `Statement` or `ResultSet`. The method returns a `SQLWarning` object that can be used in the same way as `SQLException`.

Scrollable and Updatable ResultSet

- By default, when a `Statement` is created, the corresponding `ResultSet` are read only and has a forward only access.
- You may obtain a `Scrollable` and `Updatable ResultSet` by using

```
Statement createStatement(
    int resultSetType,
    int resultSetConcurrency)
```

as follows:

```
Statement stmt = conn.createStatement(
    ResultSet.TYPE_SCROLL_SENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

- You may now, run through a `ResultSet` forward and backward (`next()` , `previous()`), go to the first, last or to a specific row. You may also update a row, insert a new row, etc.